Recipe Recommendation and Promoting Healthier Alternatives: A Comparative Study between Traditional Models and LLMs

Aníbal Fermandois [*1] **Erick Lara** [*1] **Lucas Rojas** [*1]

Abstract

This work explores the potential of large language models (LLMs), both in zero-shot and fine-tuned settings, to solve the problem of personalized recipe recommendation using structured prompting techniques. Our goal is to maximize user satisfaction while promoting healthier food choices, and we do this through a large-scale dataset of over 500k recipes and 1.4M user-recipe interactions (Food.com), evaluating 3 methodological families: baselines (including SVD), zero-shot LLM prompting, and a fine-tuned LLM. Results show that zero-shot models can be an alternative against traditional models (like SVD), while the fine-tuned model shows improvement with a 67% and a 27% decrease in the MAE and RMSE scores, respectively, over its zero-shot counterpart. These findings demonstrate that LLM-based approaches can provide a competitive or superior recommendation quality, highlighting a potential direction for an integration of health-oriented objectives into personalized food recommendation systems.

1. Introduction

When it comes to recommending items to a user, food recipe recommendation poses a unique set of challenges that makes it very different from more traditional domains such as movies or e-commerce. Beyond just capturing a user's historical preferences and explicit feedback, a recipe recommender should generally integrate strict dietary constraints, including allergies, food intolerances, cultural considerations, number of calories, and broader notions of nutritional quality. These factors introduce multi-objective requirements that typical accuracy-driven recommenders often wouldn't factor into the recommendation process. Systems optimized solely for precision may succeed at predicting what the user is likely to click on, but with recipes they frequently have a structural bias toward highly caloric, low-nutritional, or otherwise unhealthy recipes, which is an issue observed in prior work on food and menu recommendations (Trattner & Elsweiler, 2017).

Traditional approaches to address these challenges typically rely on architectures with some combination of collaborative filtering, content-based filtering, nutritional scoring mechanisms, and satisfying constraints in optimization problems. While these have been very effective, they require substantial fine-tuning and might lack flexibility when new constraints or dietary rules are introduced. Furthermore, explicitly encoding nutritional guidelines or user-level health restrictions often results in processes that don't generalize well across users with different culinary habits or health goals.

Given the recent advancements in their capabilities, Large Language Models (LLMs) have emerged as a versatile alternative for building recommender systems capable of reasoning over all kinds of information (Hou et al., 2024). Their capacity to interpret descriptions, perform multi-step reasoning, and incorporate external constraints through prompting suggests that they may serve as flexible "all-in-one" recommenders. In particular, structured prompting enables LLMs to evaluate nutritional components, classify ingredients, justify recommendations, and adhere to constraints without requiring explicit architectural modifications. The availability of increasingly smaller and efficient local models also raises the question of whether LLM-based recommenders can be not only accurate but also lightweight and deployable.

This work explores the potential of LLMs for healthy recipe recommendation by doing a comparative study between more traditional recommender models and LLM-based approaches under zero-shot and fine-tuned configurations. We analyze to what extent LLMs can simultaneously optimize user satisfaction and nutritional quality, and whether they exhibit an inherent understanding of dietary health and user

[1]Department of Computer Science, Pontificia Universidad Católica de Chile, Santiago, Chile. Correspondence to: Lucas Rojas <lucas.rojas@uc.cl>, Erick Lara <emlara@uc.cl>, Aníbal Fermandois <anibal.fermandois@uc.cl>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

taste profiles. Our study looks to determine whether LLMs can mitigate the unhealthy-recipe bias found in conventional systems while maintaining competitive accuracy, and how fine-tuning influences this balance.

2. Background

2.1. Complementary material

2.1.1. Traditional Recommendation Models

Recommender systems aim to predict user preferences for items based on historical interaction data. The most common traditional approaches include Memory-Based or Model-Based approaches for collaborative filtering. Collaborative filtering (CF) models preferences based on user-item interactions (generally reviews or ratings) and assumes that users with similar preferences in the past will behave similarly in the future, and likewise that items receiving similar feedback will appeal to similar users. Within Memory-Based. User-Based CF finds users with similar rating patterns and uses their ratings to predict missing ones, while Item-Based CF computes similarity between items and recommends items similar to those the user already liked. Memory-based CF is intuitive and relatively easy to implement, but suffers when data is sparse or lacking or when the number of users/items is very large.

Model-based CF, on the other hand, learns latent factors that represent some underlying pattern in user behavior. Latent Factor Models such as Matrix Factorization decomposes the user–item interaction matrix R into two low-rank matrices:

$$R \approx UV^T$$

where U has the latent factors of the users, and V those of the items.

Important variants include: SVD or Funk-SVD (popularized during the Netflix Prize competition and one we use as a baseline), Regularized MF, Non-Negative Matrix Factorization, Probabilistic Matrix Factorization (PMF), and Bayesian PMF.

The advantages of this approach are that it generally handles very large datasets efficiently and is robust to sparsity. More importantly, it manages to capture more complex relationships not visible to memory-based CF.

2.1.2. Large Language Models

LLMs are architectures, mostly transformer based, that are trained on massive bodies of text to learn general purpose linguistic, semantic, and reasoning capabilities. Modern LLMs with billions of parameters have demonstrated their capabilities in understanding text and reasoning even without supervision for a particular given task.

2.1.3. Parameter Efficient Fine-Tuning (LoRA)

Given that retraining and fine-tuning LLMs from the ground up is incredibly costly, Parameter-Efficient Fine-Tuning (PEFT) methods aim to adapt large pretrained models to new tasks without needing to modify all the parameters. Among these methods, Low-Rank Adaptation (LoRA) (Hu et al., 2021) is based on the observation that, during fine-tuning, the updates applied to a model's weight matrices often have an intrinsically low-dimensional structure. Instead of directly modifying the original weights, LoRA introduces a low-rank factorization that represents the update as the product of two smaller matrices. During training, the original model parameters remain frozen, and only the low-rank matrices A and B are learned, which greatly reduces the number of trainable parameters and memory usage.

Given a full weight martix W_0, LoRA modifies the updates as:

$$W = W_0 + \Delta W = W_0 + BA$$

where

$$W_0 \in \mathbb{R}^{d\times k}, \quad B \in \mathbb{R}^{d\times r}, \quad A \in \mathbb{R}^{r\times k}, \quad r \ll min(d,k)$$

This way, we can reduce the number of trainable parameters from $O(dk)$ to $O(r(d+k))$. During forward propagation, the original output layer $h = W_0x$ becomes $h = W_0x + \frac{\alpha}{r}BAx$, where α is introduced as a parameter to stabilize optimization.

Because the low-rank matrices are small and the base model remains frozen, LoRA introduces almost no additional latency and is highly efficient in terms of memory and storage. In transformer models, LoRA is typically applied to the attention/projection matrices while keeping other components frozen. This strategy has proven effective from small, local models to extremely large models such as GPT-3 (175B), where memory requirements during fine-tuning were reduced from roughly 1.2 TB to 350 GB.

For these reasons, LoRA is well-suited for scenarios requiring the adaptation of large pretrained models to new domains with limited computational budgets, as is our case.

2.2. Related works

Food recommendation has been specifically singled out for study within the field of Recommender Systems given the unique challenges it posits. Given traditional collaborative filtering and content based techniques generally focus on accuracy-oriented metrics, they tend to overlook important factors such as nutritional balance and certain constraints that need to be met for each user. To address these issues, several works have proposed a variety of solutions that aim to improve existing recommendation models and paradigms by tackling different aspects of training and evaluation (Min et al., 2020).

Initial recommender systems designed specifically for food recommendation largely focused on using collaborative filtering, content based filtering, or some hybrid of the two. CF approaches model user preference patterns from the user's history or interactions, while CBF relies on recipe details such as ingredients, nutritional content, or even cooking methods. For example, one of the earliest works by Freyne and Berkovsky (2010) shows that similarities between ingredients improves personalization beyond what is done by traditional CF algorithms. Teng et al. (2012) elaborates upon these ideas by demonstrating that modelling ingredient co-occurrence explicitly with a network can improve preference prediction. Although these methods do achieve strong results, several studies showed that focusing on accuracy alone tended to favour relatively unhealthy recipes, thus failing to take into account the user's health needs or dietary constraints (Trattner & Elsweiler, 2017).

Ever since, research has sought to expand upon these works by implementing nutritional information into the recommendation process in order to encourage healthier eating habits. Generally these can be categorized into either Health/Nutrition Aware or Knowledge-Based/Constraint-Aware methods.

The former group focuses on using quantitative nutritional attributes of the recipes, such as calories, proportions of each kind of nutrient, sugar content, etc., directly into the recommendation model. These systems do this by modifying the objective function or certain ranking criteria so that the recommendations are better suited to the users goals, whether that be reducing caloric intake or adhering to some defined nutrient target. Early work by Ge et al. (2015) proposed one of the first health-aware recommender systems by including nutritional scores into CF to discourage unhealthy recipes showing up. A later study elaborated upon this by applying many-objective optimization to balance user preferences, nutritional values, dietary diversity, and user diet patterns (Zhang et al., 2022), achieving far more balanced recommendations than classical preference-only models.

The latter group, Knowledge-Based and Constraint-Aware methods, applies structured representations of food, like ontologies, taxonomies, or food knowledge graphs to capture some relation between ingredients, nutrients, restrictions, and user profiles. Notable examples include FoodKG (Haussmann et al., 2019) that builds a large knowledge graph connecting ingredients, nutrients, and the cooking process to enable content-aware reasoning. Mckensy-Sambola et al. (2022) applies an ontology based method to create a dietary recommender that formally encodes user restrictions or characteristics with recipe attributes to be able to use logical inference to determine which recipes are best suited to the user. More recent works extend this direction towards machine learning by applying graph neural networks to encode nutritional and ingredient relationships. For example, the Nutrition-Related Knowledge Graph Neural Network (NRKG) model integrates nutrient–ingredient interactions into a graph, allowing for fine-grained reasoning about both user preferences and nutritional preferences (Ma et al., 2024). Relying on domain knowledge allows these frameworks to have strong results with constraint handling, though they do depend on expert-driven curation.

Ever since the introduction of the transformer architecture, LLMs have shown strong potential for a variety of different applications, and food related tasks are no exception. For example, KERL (Mohbat & Zaki, 2025) combines the food knowledge graphs from before with LLM reasoning, where given a user's query and constraints, the system gets relevant subgraphs, feeds them to an LLM, and outputs personalized recipe suggestions. More advanced frameworks such as MOPI-HFRS (Zhang et al., 2024) optimize both user preference, personalized healthiness, and nutritional diversity, while using an LLM to provide explanations for each recommendation. These examples support the idea that LLMs can effectively reason about food, however comparisons between LLM-based recommendation and classical collaborative/filtering-based approaches, remain rare. Seeing as LLMs generally struggle to equal these models (Hou et al., 2024), its important to study whether it's possible to maximize an LLM's performance while still promoting healthy, constraint bound recipes.

With both the benefits and drawbacks detailed in this section, we look to evaluate traditional recommenders and LLM-based models, both large scale and small local LLMs, under both preference-oriented and health-aware metrics, exploring whether LLMs inherently understand dietary health and user taste profiles.

3. Methodology

The approach employed in the development and evaluation of this investigation is divided into 3 sections:

3.1. Data Preparation and Preprocessing

The used dataset (Li, 2019) consists of 2 primary sources: recipe details and user interactions.

3.1.1. Recipe Details Data

This section of the dataset has the following statistics:

To retrieve and adapt the information of this section, we conducted the following preprocessing procedure:

- To parse the time attributes (CookTime, PrepTime, TotalTime) we first tried to parse them from the ISO 8601 format, if the operation was successful, the data was

Metric	Value
Number of recipes	522,517
Number of authors	57,178
Average rating	4.632
Rating S.D	0.642
Average review count (per recipe)	5.23
Max review count	3,063
Average calories	484.44
Calories S.D	1,397.12

Table 1. General statistics for the dataset (Recipes.csv)

Metric	Value
Number of users	271,907
Number of items	271,678
Total ratings	1,401,982
Average review count (per user)	5.16
Average rating	4.408
Rating S.D.	1.272
Max ratings by a user	8,842
Max ratings of item	2,892
Density	0.00001898

Table 2. General statistics for the dataset (Reviews.csv)

converted from seconds to minutes. If it failed, we subsequently applied regular expressions to identify common textual time formats and returned the corresponding total time in minutes.

- Nutritional attributes were converted to numeric type using the *pandas* function "to_numeric" to handle malformed entries.
- Nutritional scores were computed using a custom NutriScore function, which was adapted from the European Nutri-Score system. This system assigns a letter grade (A=5 to E=1) based on a point system where fruits, vegetables, fibers and proteins receive positive points, while calories, sugars, saturated fats and sodium receive negative points. Given that the recipes don't contain the percentage of fruits/vegetables needed for calculating the positive points, we simply allotted each recipe 1 point in this category.
- The dataset contains some recipes with missing key nutritional information, therefore, we filtered out those entries to ensure that all retained recipes had non-null nutritional data.
- Given the size of the dataset and our computational constraints, we sampled a subset of 10.000 recipes to ensure computational efficiency in the LLM experiments.

3.1.2. User Interactions Data

This section of the dataset consists of 1.401.982 user-recipe interactions, each one with 8 attributes, including UserId (as AuthorId), RecipeId, Rating (0-5 scale), and DateSubmitted.

The data (Reviews) was split randomly into 80% training and 20% testing sets using *Scikit-Surprise*'s "train_test_split" function.

In addition, both sections of the dataset were cleaned by removing duplicate recipes (based on RecipeId) and duplicate user-recipe interactions (based on ReviewId) using the *pandas* "drop_duplicates" function.

3.2. Model Architectures and Approaches

Three main approaches were explored: baseline methods (including SVD-based collaborative filtering), zero-shot LLM prompting, and fine-tuned LLMs. The objective across all methods was to generate the top-10 recipe recommendations per user, balancing relevance and nutritional quality, which is measured using NutriScore.

3.2.1. Baselines

The following baselines were employed:

- Random Recommender: Randomly selects 10 unseen recipes from the candidate pool.
- Most Popular Recommender: Recommends the globally most-rated recipes.
- User Mean Recommender: Predicts a user's rating for an item by assigning the mean of all ratings previously given by that user.
- Collaborative Filtering (SVD): Implemented using the SVD model from *sckit-surprise*. The model was trained using default hyperparameters.

3.2.2. Zero-Shot(s)

For the Zero-Shot LLM Prompting approach, large language models were queried directly to estimate user-item relevance without any task-specific training. Several models were evaluated, such as Gemini 2.0 flash lite via the *google-generative-ai* API, local Gemma 3 (via *Ollama*) and Phi models (2 and 3 mini, via *Ollama*). A structured prompt was used to obtain a numerical rating prediction:

"You are predicting how much each user would rate a recipe. Output only a single whole number between 1 and 5 (inclusive) for each user, one per line. Do not use decimals or fractions. Consider the WHO Health Score into the recommendation, giving preference to healthier recipes. Respond ONLY with one rating per line in the same order."

For each test interaction, the user's most recent five ratings were combined with the candidate recipe's metadata and fed into the model. The predicted rating was extracted with a regex-based parser (findall(r"\d+", text)), clamping outputs outside the 1-5 range.

3.2.3. FINE-TUNED

Finally, for the Fine-Tuned LLM approach, we adapted a smaller language model on the rating prediction task through supervised fine-tuning, where the data was split into training and validation sets using an 80/20 ratio. The base model was Microsoft Phi-2, selected for its efficiency and suitability for local hardware, given the computational constraints. The datasets were reformatted into JSONL pairs of (*user_history* + *recipe_details*) and *true_rating*, with the following prompt:

"You are a model that predicts how much a user will rate a recipe. Respond ONLY with a single number from 1 to 5 (no words). Higher numbers mean the user will like the recipe more. You should also favor recipes with a healthier NutriScore. User ID: {uid} User History: {history_text} Recipe: - Name: {recipe['Name']} - Health Score (NutriScore): {recipe['NutriScore']} Final answer (1–5):"

Fine-tuning was performed using PEFT with LoRA (rank=8, α = 16), targeting the q_proj, k_proj, v_proj and dense modules. Training was carried out with *TrainingArguments*, from *Transformers*, using a batch size of 1 per device, gradient accumulation of 4 steps (effective batch size = 4), 1 training epoch, learning rate of 1×10^{-4}, FP16 mixed precision, and periodic logging and evaluation every 10-100 steps. After training, early stopping was applied based of validation performance, and the learned LoRA weights were merged into the base model to obtain the final fine-tuned checkpoint. Finally, during inference, the predictions were generated in batches following the same structure as the zero-shot prompting setup, but benefiting from the specialization acquired through fine-tuning.

3.3. Evaluation

Evaluation focused on recommendation quality and health integration, using the test split described earlier.

3.3.1. RANKING METRICS

Ranking performance was evaluated using standard top-N metrics computed through custom functions. Precision@10 and Recall@10 measured the proportion of relevant items (ratings ≥ 4), among the top-10 recommendations and the proportion of relevant items successfully retrieved. Meanwhile, NDCG@10 (Normalized Discounted Cumulative Gain) captured ranking quality by weighting items according to their position and comparing against the ideal ordering derived from relevant items.

3.3.2. RATING PREDICTION METRICS

Rating prediction performance was assessed for the models capable of generating explicit rating estimates, such as Random, SVD, zero-shot LLM prompting, and fine-tuned LLMs. For these methods, RMSE (Root Mean Squared Error) and MAE (Mean Absolute Error) were computed. However, these metrics do not apply to the "Most Popular" baseline, as it produces only ranked recommendations without predicting numerical ratings.

3.3.3. HEALTH METRIC

To assess the nutritional quality of recommendations, we make use of MeanHealth@10, defined as the average NutriScore value among the top-10 recommendations for each user, this score was computed using the scoring function described earlier, which assigns a letter grade (A-E) based on negative components, such as calories, sugar, saturated fat and sodium), and positive components, like fiber and protein. Then, these letter grades were converted into numeric values, where; A=5, B=4, C=3, D=2 and E=1, higher values indicate healthier recommended recipes.

4. Results

We evaluate all models across three main dimensions: Rating prediction accuracy, top-10 ranking quality and health awareness of the recommended recipes.

4.1. Rating prediction results

Table 3 shows MAE and RMSE on the test set. The traditional recommendation method (SVD) already outperforms every zero-shot LLM in rating prediction. However, the fine-tuned version of Phi-2.7B has a substantial better result, reducing MAE from 1.808 to 0.582 (a 67.8% improvement) and RMSE from 1.899 to 1.386 (a 27% improvement). While its RMSE is not better than SVD, fine-tuning closes a significant portion of the performance gap between LLMs and classical collaborative filtering. These results confirm that when the goal is a precise numerical rating prediction, an efficient fine-tuning of explicit rating data can be superior to zero-shot prompting.

4.2. Ranking@10 results

Table 4 presents Precision@10, Recall@10 and NDCG@10. Contrary to rating prediction, zero-shot LLMs dominate the ranking task, especially Gemini 2.0 FL (Flash Lite), which achieves the best performance across all three metrics, improving Precision@10 by 77%, Recall@10 by 9% and NDCG@10 by 41% over SVD. Notably, the fine-tuned Phi-

Model	MAE	RMSE
Baseline		
Random	1.921	2.367
Most Popular	–	–
SVD	0.744	1.223
Zero-Shot		
Gemini 2.0 FL	0.914	1.430
Phi-3 Mini 3.8B	1.349	1.938
Phi-2.7B	1.808	1.899
Fine-Tuned		
Phi-2.7B	**0.582**	**1.386**

Table 3. Rating prediction results (MAE and RMSE).

Model	MeanHealth@10
Baseline	
Random	**3.3**
Most Popular	1.5258
SVD	2.4806
Zero-Shot	
Gemini 2.0 FL	2.5931
Phi-3 Mini 3.8B	2.5585
Phi-2.7B	2.5664
Fine-Tuned	
Phi-2.7B	2.5635

Table 5. Health Metric results (MeanHealth@10).

2.7B does not surpass Gemini 2.0 FL, and performs on par with the much bigger Phi-3 Mini 3.8B and the non-fine-tuned Phi-2.7B.

Model	Prec@10	Rec@10	NDCG@10
Baseline			
Random	0	0	0
Most Popular	0.004	0.021	0.013
SVD	0.084	0.429	0.434
Zero-Shot			
Gemini 2.0 FL	**0.149**	**0.468**	**0.612**
Phi-3 Mini 3.8B	0.143	0.422	0.561
Phi-2.7B	0.144	0.417	0.559
Fine-Tuned			
Phi-2.7B	0.144	0.417	0.559

Table 4. Ranking Results (@10).

4.3. Health results

Table 5 reports the MeanHealth@10 metric, which corresponds to the average NutriScore of the top-10 recommended recipes, where higher values indicate the presence of healthier recommendations on average. The random baseline yields the healthiest recommendations with a score of 3.3, followed by the result of Gemini 2.0 FL with a MeanHealth@10 of 2.5931, which is 21.4% worse than the random baseline. In contrast, the "Most Popular" approach suffers from a clear unhealthy bias, achieving a MeanHealth@10 of 1.5258, the worst among all models, followed by the result of SVD with 2.4806, which is 38.49% better than Most Popular, representing an even larger gap than that of Gemini 2.0 flash lite and the best result.

5. Discussion

Our analysis reveals a distinct relationship between ranking quality and health promotion in recipe recommendation. A central finding is that these two metrics function as orthogonal, often competing, objectives most clearly illustrated by the baseline models: the Random recommender achieved the highest health scores but the lowest ranking utility, whereas the "Most Popular" baseline maximized engagement probability at the cost of providing the least healthy recommendations. This confirms the "unhealthy bias" inherent in popularity-driven systems, where users consistently prefer lower-nutritional-value options when available.

In this landscape, Large Language Models (LLMs) demonstrated a unique capacity for multi-objective reasoning. Specifically, Gemini 2.0 Flash Lite effectively bridged the gap, achieving the highest ranking performance (NDCG@10) while maintaining nutritional scores significantly superior to popularity-based baselines. This high performance in multi-objective reasoning seems to be related to its relatively higher model dimension, though the exact value has not been published or publicly disclosed.

Regarding the efficacy of fine-tuning, our results present a more nuanced picture. Fine-tuning the Phi-2 model yielded substantial improvements in rating prediction accuracy, reducing MAE by nearly 68% compared to Zero-Shot methods. However, this predictive precision did not translate into superior Top-N ranking performance, where Zero-Shot prompting remained dominant.

Finally, we observed a plateau in ranking metrics across the evaluated LLMs, with Gemini 2.0 Flash Lite being an outlier. We attribute this to the high density of positive ratings in the dataset (average rating 4.63). In such a skewed environment, models struggle to differentiate between "good" and "excellent" items, suggesting that LLM differentiation might be more pronounced in datasets with higher variance in user feedback.

As for future works, a good starting point would be to test these evaluation and training pipelines on more varied data, with recipes that ideally include the fruit/vegetable/nut percentage metric to be able to accurately calculate the NutriScore. As for the evaluation itself, testing the LLMs with different prompts would be a valuable area of study, seeing

as our focus was solely on optimizing an overall "health score". Testing the capabilities of LLMs to process user profiles and even prompted instructions for what each user explicitly wants would be more relevant to real world use cases and scenarios. Another logical direction is to use multimodal LLMs that can process recipe images together with the textual descriptions and nutritional facts from the dataset. This would allow the model to reason about visual cues, portion sizes, etc., that are not captured in text alone. Future work could also evaluate the safety of LLM generated recommendations (especially if given harmful dietary advice) and potentially incorporate embeddings with these constraints and preferences derived from CF or MF as a hybrid approach.

6. Conclusions

This study highlights the potential of LLMs to address the complex challenge of personalized, health-aware food recommendation. Our results demonstrate that while traditional collaborative filtering approaches prioritize accuracy, often at the expense of nutrition, LLMs possess an inherent capability for multi-objective reasoning that can better balance user satisfaction with dietary health. Our research shows that state of the art models like Gemini 2.0 Flash Lite exhibit higher multi-objective reasoning than the rest.

We conclude that zero-shot prompting offers a highly competitive, low commitment alternative to traditional baselines, particularly for ranking tasks. Meanwhile, Parameter-Efficient Fine-Tuning (PEFT) proves superior for explicit rating prediction tasks, offering a path to high-precision systems. Future work should focus on testing these architectures on more diverse datasets to mitigate ceiling effects and further exploring how structured prompting can enforce strict dietary constraints without compromising recommendation utility.

Software and Data

The code corresponding to our findings can be found in the following repository: https://github.com/LucasReds/ProyectoRecSys

References

Freyne, J. and Berkovsky, S. Intelligent food planning: personalized recipe recommendation. In *Proceedings of the 15th International Conference on Intelligent User Interfaces*, IUI '10, pp. 321–324, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781605585154. doi: 10.1145/1719970.1720021. URL https://doi.org/10.1145/1719970.1720021.

Ge, M., Ricci, F., and Massimo, D. Health-aware food recommender system. In *Proceedings of the 9th ACM Conference on Recommender Systems*, RecSys '15, pp. 333–334, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450336925. doi: 10.1145/2792838.2796554. URL https://doi.org/10.1145/2792838.2796554.

Haussmann, S., Seneviratne, O., Chen, Y., Ne'eman, Y., Codella, J., Chen, C.-H., Mcguinness, D., and Zaki, M. *FoodKG: A Semantics-Driven Knowledge Graph for Food Recommendation*, pp. 146–162. 10 2019. ISBN 978-3-030-30795-0. doi: 10.1007/978-3-030-30796-7_10.

Hou, Y., Zhang, J., Lin, Z., Lu, H., Xie, R., McAuley, J., and Zhao, W. X. Large language models are zero-shot rankers for recommender systems, 2024. URL https://arxiv.org/abs/2305.08845.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models, 2021. URL https://arxiv.org/abs/2106.09685.

Li, S. Food.com recipes and interactions, 2019. URL https://www.kaggle.com/dsv/783630.

Ma, W., Li, M., Dai, J., Ding, J., Chu, Z., and Chen, H. Nutrition-related knowledge graph neural network for food recommendation. *Foods*, 13(13), 2024. ISSN 2304-8158. doi: 10.3390/foods13132144. URL https://www.mdpi.com/2304-8158/13/13/2144.

Mckensy-Sambola, D., Rodríguez-García, M. , García-Sánchez, F., and Valencia-García, R. Ontology-based nutritional recommender system. *Applied Sciences*, 12 (1), 2022. ISSN 2076-3417. doi: 10.3390/app12010143. URL https://www.mdpi.com/2076-3417/12/1/143.

Min, W., Jiang, S., and Jain, R. Food recommendation: Framework, existing solutions, and challenges. *IEEE Transactions on Multimedia*, 22(10):2659–2671, 2020. doi: 10.1109/TMM.2019.2958761.

Mohbat, F. and Zaki, M. J. KERL: Knowledge-enhanced personalized recipe recommendation using large language models. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 19125–19141, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.938. URL https://aclanthology.org/2025.acl-long.938/.

Teng, C.-Y., Lin, Y.-R., and Adamic, L. A. Recipe recommendation using ingredient networks, 2012. URL https://arxiv.org/abs/1111.3919.

Trattner, C. and Elsweiler, D. Investigating the healthiness of internet-sourced recipes: Implications for meal planning and recommender systems. In *Proceedings of the 26th International Conference on World Wide Web*, WWW '17, pp. 489–498, Republic and Canton of Geneva, CHE, 2017. International World Wide Web Conferences Steering Committee. ISBN 9781450349130. doi: 10.1145/3038912.3052573. URL https://doi.org/10.1145/3038912.3052573.

Zhang, J., Li, M., Liu, W., Lauria, S., and Liu, X. Many-objective optimization meets recommendation systems: A food recommendation scenario. *Neurocomputing*, 503:109–117, 2022. ISSN 0925-2312. doi: https://doi.org/10.1016/j.neucom.2022.06.081. URL https://www.sciencedirect.com/science/article/pii/S0925231222008128.

Zhang, Z., Wang, Z., Ma, T., Taneja, V. S., Nelson, S., Le, N. H. L., Murugesan, K., Ju, M., Chawla, N. V., Zhang, C., and Ye, Y. Mopi-hfrs: A multi-objective personalized health-aware food recommendation system with llm-enhanced interpretation, 2024. URL https://arxiv.org/abs/2412.08847.

A. Appendix

```
# Negative points tables
energy_points = [80,160,240,320,400,480,560,640,720,800]
sugar_points = [4.5,9,13.5,18,22.5,27,31,36,40,45]
sat_fat_points = [1,2,3,4,5,6,7,8,9,10]
salt_points = [90/400, 180/400, 270/400, 360/400, 450/400, 540/400, 630/400, 720/400,
    810/400, 900/400]

# Positive points tables
fiber_points = [0.7, 1.4, 2.1, 2.8, 3.5]
protein_points = [1.6, 3.2, 4.8, 6.4, 8.0]

def score_from_thresholds(value, thresholds):
    pts = 0
    for t in thresholds:
        if value > t:
            pts += 1
    return pts

def compute_nutriscore(row):
    # NEGATIVE POINTS
    A = score_from_thresholds(row["Calories"], energy_points)
    B = score_from_thresholds(row["SugarContent"], sugar_points)
    C = score_from_thresholds(row["SaturatedFatContent"], sat_fat_points)

    salt = row["SodiumContent"] / 400
    D = score_from_thresholds(salt, salt_points)

    N = A + B + C + D

    # POSITIVE POINTS
    # No fruit/veg/nut % available, assume median
    E = 1

    F = score_from_thresholds(row["FiberContent"], fiber_points)
    G = score_from_thresholds(row["ProteinContent"], protein_points)

    if N >= 11:
        G = 0

    P = E + F + G

    total = N - P
    if total <= -1:
        return "A"
    elif total <= 2:
        return "B"
    elif total <= 10:
        return "C"
    elif total <= 18:
        return "D"
    else:
        return "E"
```